```
# IMPORTANT: SOME KAGGLE DATA SOURCES ARE PRIVATE
# RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES.
import kagglehub
kagglehub.login()
```

```
# IMPORTANT: RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES,
# THEN FEEL FREE TO DELETE THIS CELL.
# NOTE: THIS NOTEBOOK ENVIRONMENT DIFFERS FROM KAGGLE'S PYTHON
# ENVIRONMENT SO THERE MAY BE MISSING LIBRARIES USED BY YOUR
# NOTEBOOK.

rray11212_image_for_prediction_path = kagglehub.dataset_download('rray11212/image-for-predi
rray11212_blood_group_prediction_cnn_keras_default_1_path = kagglehub.model_download('rray1

print('Data source import complete.')
```

```
import os
print(os.listdir('/kaggle/input/models/rray11212/blood-group-prediction-cnn/keras/default/1
```

```
['blood_group_prediction_model.h5']
```

```
#Import the model
import tensorflow as tf

model = tf.keras.models.load_model('/kaggle/input/models/rray11212/blood-group-prediction-c
```

```
2026-02-13 07:41:44.243655: E external/local_xla/xla/stream_executor/cuda/cuda_platform.cc:
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `moc
```

```
#A+ test image
import os
print(os.listdir('/kaggle/input/image-for-prediction'))
```

```
['cluster_0_1001.bmp']
```

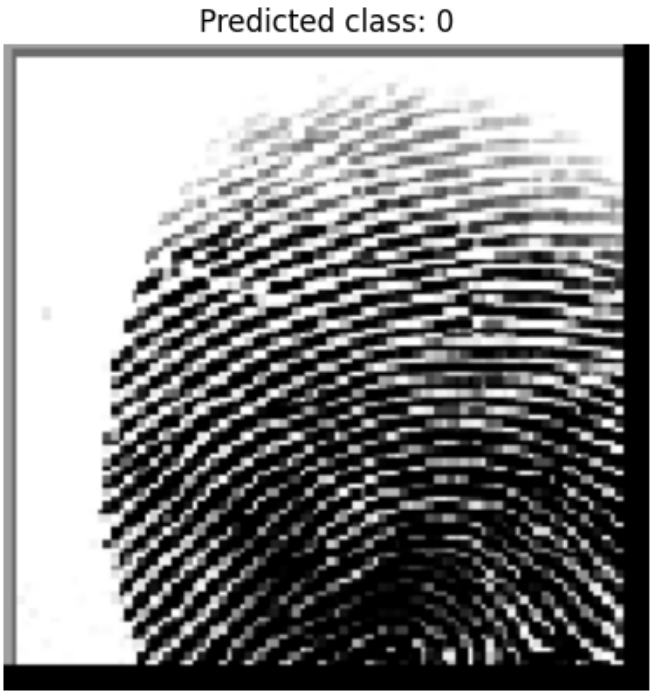
```
#load and preprocess the image
from tensorflow.keras.preprocessing import image
import numpy as np

img_path = '/kaggle/input/image-for-prediction/cluster_0_1001.bmp'
img = image.load_img(img_path, target_size=(150, 150))
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0) / 255.0
```

```
#predict using the model
predictions = model.predict(img_array)
predicted_class = np.argmax(predictions, axis=1)[0]
print("Predicted class:", predicted_class)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 226ms/step
Predicted class: 0
```

```
#visualize for class 0 i.e A+
import matplotlib.pyplot as plt
```

```
plt.imshow(img)
plt.title(f"Predicted class: {predicted_class}")
plt.axis('off')
plt.show()
```




```
#Hence , correctly predicted A+ for this test image.
```

```
# Find the last Conv2D layer in the model
last_conv_layer = None
for layer in model.layers[::-1]:
    if isinstance(layer, tf.keras.layers.Conv2D):
        last_conv_layer = layer.name
        break

print("Last conv layer:", last_conv_layer)
```

```
Last conv layer: conv2d_2
```

```
_ = model(img_array)
```

```
import tensorflow as tf

# Get the original layers from your loaded model
original_layers = model.layers

# Rebuild the model with an explicit Input layer
new_model = tf.keras.Sequential()
new_model.add(tf.keras.layers.InputLayer(shape=(150, 150, 3)))
for layer in original_layers:
    new_model.add(layer)

# Call the model once to build layer outputs
_ = new_model(img_array)
```

```
_ = model.predict(img_array)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 47ms/step
```

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

# 1. Get config and weights from your loaded model
config = model.get_config()
weights = model.get_weights()

# 2. Rebuild as functional model
inputs = tf.keras.Input(shape=(150, 150, 3))
x = inputs
for layer_config in config['layers'][1:]:  # skip the InputLayer if present
    layer = tf.keras.layers.deserialize(layer_config)
    x = layer(x)
functional_model = tf.keras.Model(inputs, x)
functional_model.set_weights(weights)

# 3. Call the model once to build layer outputs
_ = functional_model(img_array)

# 4. Grad-CAM function
def make_gradcam_heatmap(img_array, model, last_conv_layer_name, pred_index=None):
    grad_model = tf.keras.models.Model(
        [model.input], [model.get_layer(last_conv_layer_name).output, model.output]
    )
    with tf.GradientTape() as tape:
        conv_outputs, predictions = grad_model(img_array)
        if pred_index is None:
            pred_index = tf.argmax(predictions[0])
        class_channel = predictions[:, pred_index]
    grads = tape.gradient(class_channel, conv_outputs)
    pooled_grads = tf.reduce_mean(grads, axis=(0, 1, 2))
    conv_outputs = conv_outputs[0]
    heatmap = conv_outputs @ pooled_grads[..., tf.newaxis]
    heatmap = tf.squeeze(heatmap)
    heatmap = tf.maximum(heatmap, 0) / tf.math.reduce_max(heatmap)
    return heatmap.numpy()

# 5. Generate Grad-CAM heatmap
heatmap = make_gradcam_heatmap(img_array, functional_model, 'conv2d_2', pred_index=predict

# 6. Display heatmap on image
plt.imshow(img)
plt.imshow(heatmap, cmap='jet', alpha=0.5)
plt.title('Grad-CAM')
plt.axis('off')
plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/models/functional.py:241: UserWarning: Th
Expected: ['keras_tensor_139']
Received: inputs=Tensor(shape=(1, 150, 150, 3))
  warnings.warn(msg)
```